ORCA VS Code Extension — Final Plan

Revised from the original proposal after review against the actual codebase. The direction stands: expose the pipeline as a clean library, build native VS Code surfaces on top. The revisions: several "prerequisite refactors" are already done, four real gaps were missing, and the sequencing now starts from what exists.

Corrections to the original proposal

Original claim	Reality
`review.ts` "prompts via stdin/stdout" and needs refactor	`reviewPlan` already takes injectable `ReviewIO` streams (tests drive it that way); only the *callback* seam is new
Logging "should go through an injectable sink"	Already true — `Logger` is an interface on `RunContext`; an extension implements it onto an OutputChannel with zero core work
`runPipeline`/`executeRun` both live in `core/pipeline.ts`	`executeRun` is CLI-layer (process streams, exit codes). The extension calls `runPipeline` directly
"npm link or workspace reference"	Repo is deliberately single-package; adopting npm workspaces is a conscious decision, made below

Gaps the original proposal missed (now in scope)

1. **Library surface.** `package.json` exports only `dist/core/pipeline.js`. The extension also needs `createRunContext`, plan types, `renderDiagram`, `loadSettings`, `AIRouter`/`aiFor`. → Add subpath exports.
2. **Secrets.** Both AI providers read keys from `process.env` directly. `vscode.SecretStorage` needs a credentials-provider seam in core (mutating the shared extension host's `process.env` is not acceptable).
3. **Cancellation.** The pipeline has no `AbortSignal`. A webview closed mid-approval or a cancel button during a docker build must be able to stop the run — and a disposed approval panel must reject its pending promise or the pipeline hangs.
4. **Webview assets.** Bundle `mermaid.js` into `media/` — webview CSP and offline use both rule out a CDN.

Deferred (explicitly out of v1): live streaming of docker output. `runOnce` reports per-phase; the heal log updates per phase, which is enough.

Phase 0 — prerequisites (no extension code)

- [ ] One real `orca run examples/mern-sample` on a docker machine (the build path is currently mock-verified only). Fix whatever it surfaces.
- [ ] Publish `orca-cli@0.1.0` to npm; tag `v0.1.0`.

Phase 1 — core embeddability (ships as `orca-cli@0.2.0`, CLI unchanged)

- [ ] `PipelineIO.approvePlan?: (plan: ServicePlan) => Promise<ReviewOutcome>` — optional hook; default remains `reviewPlan`. CLI untouched, tests extended.
- [ ] Subpath exports: `orca-cli` (pipeline + context), `orca-cli/plan`, `orca-cli/config`, `orca-cli/ai`. Verified by an import-smoke test.
- [ ] Credentials provider: providers resolve keys through an injectable `resolveKey(name)` (default: `process.env`), carried on `RunContext` alongside `ai`.
- [ ] `AbortSignal` on `RunOptions` → threaded to clone (`simple-git`), docker (execa `signal`), AI fetch/SDK calls, and checked between stages. Aborted runs tear down (`composeDown`) and clean the workspace like any failure.

Phase 2 — workspace conversion

- [ ] npm workspaces: root package stays `orca-cli` (publishable, unchanged layout); add `"workspaces": ["vscode-extension"]`. Update README's single-package note. CI builds both.

Phase 3 — bare-bones extension (first demoable milestone)

- [ ] `vscode-extension/` manifest: name `orca-vscode`, one command **"ORCA: Preview Repo (Dry Run)"**, activation on command.
- [ ] Command flow: pick folder (default: current workspace folder) → `createRunContext` (keyless is fine — dry run needs no keys/docker) → `runPipeline` with `dryRun + autoApprove`, `Logger` → OutputChannel, preview text → OutputChannel.
- [ ] Runs under F5 (Extension Development Host). This proves all the wiring with the least possible UI.

Phase 4 — plan webview (the headline feature, natively)

- [ ] Webview panel: bundled mermaid renders the diagram, service table beside it, Approve / Reject buttons. `postMessage` both ways.
- [ ] Wire it as the `approvePlan` hook; panel disposal rejects → pipeline takes its normal rejection path.
- [ ] Replace `autoApprove` in the Phase 3 command with the real gate.

Phase 5 — settings & secrets

- [ ] Contributed `configuration` (model, cost cap, heal attempts, providers) mapped onto `RunOptions`/`loadSettings` values; precedence stays flag/UI > env > default.
- [ ] API keys via `SecretStorage` through the Phase 1 credentials provider; "ORCA: Set API Key" command; never written to settings.json.

Phase 6 — full run + heal log

- [ ] **"ORCA: Run Repo"**: full pipeline; per-phase progress via `withProgress` + OutputChannel; heal `DiagnosisEntry`s rendered as they are appended; cancel button → `AbortSignal`.
- [ ] Success state: notification with service URLs + "Open in browser" / "Stop stack" actions.

Phase 7 — packaging & release

- [ ] `vsce package`; marketplace listing (README, icon, demo GIF of the approval webview).
- [ ] CI job: build + test extension alongside core; smoke = Phase 3 command against `examples/mern-sample` in the test harness.

Testing strategy

Core changes (Phases 1–2) are covered by the existing hermetic vitest suite, extended per feature — no keys, no docker, no VS Code needed. The extension keeps its logic thin (all decisions stay in core); webview/host glue gets a minimal `@vscode/test-electron` smoke, not a parallel test universe.